

TP : Diagnostic de la Rouille Polysora sur les Feuilles de Maïs à Madagascar

Rakotoarimalala Tsinjo Tony

Objectifs pédagogiques :

1. Comprendre la chaîne complète de traitement de données : des images brutes à la prédiction.
2. Implémenter "*from scratch*" une variante algorithmique d'arbre de décision basée sur une métrique personnalisée de pureté : **Max-Minority**.
3. Manipuler les hyperparamètres d'un modèle *Random Forest* et analyser l'importance des variables dans un contexte agronomique réel.

1 Contexte du TP

La sécurité alimentaire à Madagascar dépend fortement de la culture du maïs. Cependant, les plantations sont régulièrement menacées par des maladies fongiques, notamment la **Rouille Polysora** (*Puccinia polysora*), particulièrement dévastatrice dans les zones chaudes et humides de l'île.

En tant que futurs informaticiens et scientifiques des données, votre objectif est de concevoir un système d'aide au diagnostic pour les techniciens agricoles locaux. À partir de photos de feuilles de maïs prises sur le terrain (à vous de composer et de trouver ces données), votre algorithme doit classer la feuille en deux catégories : **Saine (0)** ou **Malade (1)**.

Vous disposez d'un jeu de données d'images réparti en deux dossiers :

- `dataset/saines/` : Photos de feuilles de maïs saines.
- `dataset/malades/` : Photos de feuilles présentant des pustules orangées/brunâtres de rouille.

2 Partie 1 : Feature Engineering — Du Pixel aux Caractéristiques

Les arbres de décision ne peuvent pas traiter directement des matrices de pixels bruts de manière efficace. Nous devons d'abord extraire des descripteurs numériques pertinents à l'aide de techniques de traitement d'images.

Travail à faire :

À l'aide des bibliothèques `OpenCV` (ou `scikit-image`) et `NumPy`, écrivez un script Python qui parcourt les dossiers d'images et extrait, pour chacune, un vecteur de caractéristiques (*features*) :

1. **Extraction de couleur (Espace HSV) :**
 - Convertissez l'image de RGB à HSV via la fonction `cv2.cvtColor(img, cv2.COLOR_BGR2HSV)`. L'espace HSV permet d'isoler facilement les couleurs indépendamment des variations de luminosité.
 - Définissez un masque de couleur pour isoler les teintes "rouille" (marrons / jaunes / oranges).
 - Calculez la caractéristique X_1 : `pct_rouille` (Nombre de pixels de rouille / Nombre total de pixels de la feuille).

2. Extraction de texture et rugosité (Filtre de Sobel) :

- Les pustules de rouille créent une irrégularité tactile et visuelle sur la feuille. Appliquez un filtre de Sobel pour détecter les contours et les variations brusques d'intensité.
- Calculez la caractéristique X_2 : **rugosite** (Variance ou moyenne de l'intensité des gradients de Sobel).

3. Extraction d'un feature de votre choix :

- ajouter une variable supplémentaire de votre choix
- cette variable doit être justifiée par une intuition agronomique/informatique
- Cette variable doit être personnelle (différente pour chaque étudiant)

Livrable intermédiaire : Un tableau Pandas `DataFrame` contenant la structure suivante :

[`ID_Image` | `pct_rouille` | `rugosite` | `votre_variable` | `label_malade`] (où `label_malade` vaut 0 ou 1).

3 Partie 2 : Le Défi Algorithmique — L'Indice "Max-Minority"

Dans les cours théoriques, le choix du meilleur découpage (*split*) d'un nœud se fait via l'Indice de Gini ou l'Entropie. Dans ce TP, vous allez implémenter et tester une autre approche algorithmique : la métrique **Max-Minority**.

3.1 Fondement Mathématique

L'objectif de cette métrique est de mesurer la pureté d'un nœud en évaluant sa capacité à isoler ou à écraser la classe minoritaire de manière linéaire.

Pour un nœud t contenant N individus, la pureté $P(t)$ est définie par la proportion de la classe majoritaire :

$$P(t) = \max_{c \in \{0,1\}} \left(\frac{n_c}{N} \right)$$

Exemple : Si un nœud contient 90% de feuilles saines et 10% de feuilles malades, sa pureté est $P(t) = \max(0.9, 0.1) = 0.9$. Si le nœud est parfaitement mélangé (50/50), alors $P(t) = 0.5$.

3.2 Algorithme de calcul du meilleur split

Pour une variable continue (ex : `pct_rouille`), vous devez implémenter la recherche du seuil optimal par balayage (*brute-force* optimisé) :

1. Triez les valeurs de la variable de manière croissante.
2. Pour chaque seuil candidat s (calculé comme le milieu entre deux valeurs consécutives uniques), séparez virtuellement les données en deux sous-groupes : *Gauches* (G) et *Droites* (D).
3. Calculez la **Pureté Pondérée du Split** :

$$P_{split} = \frac{|G|}{N} P(G) + \frac{|D|}{N} P(D)$$

Où $|G|$ et $|D|$ représentent le nombre d'éléments à gauche et à droite, et N le nombre total d'éléments dans le nœud parent.

Travail à faire :

Écrivez une fonction Python `trouver_meilleur_split(X_column, y)` qui teste tous les seuils possibles pour une variable donnée et retourne le seuil s qui maximisera P_{split} , ainsi que la valeur de cette pureté.

```
1 def trouver_meilleur_split(X_column, y):
2     # X_column : array-like des valeurs de la variable a tester
3     # y : array-like des labels correspondants (0 ou 1)
4
5     # etape 1 : Trier les donnees par X_column
6     # etape 2 : Initialiser les variables pour suivre le meilleur seuil
7     # etape 3 : Parcourir les seuils candidats et calculer P_split pour
8     # etape 4 : Retourner le seuil optimal et sa purete associee
```

Listing 1 – Prototype de la fonction de split

4 Partie 3 : Arbres et Forêts "From Scratch" vs Scikit-Learn

Cette partie est dédiée à la construction récursive de vos modèles, à l'utilisation des solutions clés en main de la bibliothèque `scikit-learn`, et à la confrontation de leurs performances.

4.1 Implémentation "From Scratch" (Fait Maison)

En utilisant la fonction `trouver_meilleur_split` développée à la Partie 2, vous devez programmer vos propres structures prédictives :

1. **L'Arbre de Décision Max-Minority** : Développez une classe ou une fonction récursive `build_tree(X, y, depth, max_depth)` qui construit l'arbre de décision. Le processus doit s'arrêter si le nœud est 100% pur ($P(t) = 1$) ou si la profondeur maximale `max_depth` est atteinte.
2. **Le Random Forest Max-Minority** : En combinant plusieurs de vos arbres personnalisés, créez une version simplifiée de Forêt Aléatoire. Votre algorithme devra implémenter :
 - Le *Bagging* (sous-échantillonnage des lignes avec remplacement via `np.random.choice`).
 - La prédiction finale par vote majoritaire (agrégation des résultats de chaque arbre).

4.2 Utilisation des modèles de Scikit-Learn

Afin de valider votre approche, vous allez maintenant instancier les modèles industriels correspondants. Séparez au préalable votre tableau de caractéristiques en sous-ensembles d'apprentissage (80%) et de test (20%).

- Entraînez un modèle `DecisionTreeClassifier` (avec le critère Gini).
- Entraînez un modèle `RandomForestClassifier` (`n_estimators=100`).

4.3 Comparaison et Analyse Critique

Réalisez une étude comparative rigoureuse des quatre configurations suivantes :

1. Votre Arbre "From Scratch" (Métrique Max-Minority).
2. Votre Random Forest "From Scratch" (Métrique Max-Minority).

3. L'Arbre de Scikit-Learn (Métrique Gini).
4. Le Random Forest de Scikit-Learn (Métrique Gini).

Questions et analyses attendues :

1. **Évaluation quantitative :** Calculez et affichez sous forme de tableau comparatif l'*Accuracy* (exactitude), la *Précision* et le *Rappel (Recall)* sur le jeu de test pour chacun des quatre modèles.
2. **Analyse du comportement :** Votre algorithme personnalisé obtient-il des performances proches de celles de `scikit-learn`? Justifiez pourquoi la Forêt Aléatoire (qu'elle soit maison ou officielle) améliore systématiquement la robustesse par rapport à un arbre unique.
3. **Discussion agronomique (Madagascar) :** Dans le contexte agricole malgache, une erreur de diagnostic peut avoir de lourdes conséquences. Si le modèle manque une feuille malade (**Faux Négatif**), l'épidémie se propage et détruit les récoltes. S'il se trompe sur une feuille saine (**Faux Positif**), l'agriculteur gaspille des produits de traitement coûteux.

Au vu de vos matrices de confusion, quel modèle recommandez-vous officiellement de déployer sur le terrain pour les techniciens agricoles? Argumentez votre choix en exploitant les notions de *Précision* et de *Rappel*.

5 Partie 4 : Déploiement d'une Application Web avec Streamlit

Afin de rendre votre modèle accessible aux techniciens agricoles sur le terrain à Madagascar, vous allez développer une application web légère en utilisant le framework **Streamlit**.

5.1 Cahier des charges de l'Application Web

Créez un fichier Python nommé `app.py`. Votre application devra être découpée en deux fonctionnalités majeures :

1. **Module d'Upload et de Prédiction en Temps Réel :**
 - L'utilisateur peut téléverser une image de feuille de maïs (`.png`, `.jpg`, `.jpeg`) via un composant `st.file_uploader`.
 - Dès le téléversement, l'application affiche l'image de la feuille sur l'interface.
 - En arrière-plan, l'application exécute vos fonctions de la **Partie 1** pour extraire en temps réel le `pct_rouille` et la `rugosite` de cette nouvelle image.
 - Le modèle chargé (`pickle.load`) effectue la prédiction et affiche le résultat de manière claire et visuelle à l'écran (ex : Une alerte rouge "**ATTENTION : Feuille Malade (Rouille Detected)**" ou un message vert "**Feuille Saine**").
2. **Galerie d'Historique des Détections :**
 - Créez un mécanisme permettant de mémoriser localement les images analysées (par exemple, en les copiant dans un dossier temporaire `uploads/`).
 - À l'aide des colonnes Streamlit (`st.columns`), concevez une galerie visuelle affichant les miniatures des photos précédemment analysées accompagnées du diagnostic calculé par l'algorithme sous chaque image.

Travail à faire et démonstration :

Lancez votre serveur en local à l'aide de la commande `streamlit run app.py`. Testez votre application en téléversant de nouvelles images de test et assurez-vous du bon fonctionnement de la galerie d'historique.